ILC Lighting App

Session orientation for Manuel San Ramon · July 2026

THE PROJECT

Internal tool for ILC Studios (lighting consultancy). Produces conceptual and design budgets, manages projects, and runs a CRM pipeline. Live at `https://app.ilcstudios.com`, auto-deploys from `main` via AWS Amplify.

Stack: Next.js 14 App Router, Tailwind CSS, Prisma 5.22 (SQLite locally / PostgreSQL in prod), JWT auth, Amplify SSR.

THE TEAM

Name	Role
Mike Kershner	Founder and lead dev. Pushes directly to main.
Sean McCauley	Manny's reviewer. Merges all Manny's PRs.
Manny (you)	Part-time dev ~10 hr/week. Always works on manny/* branches.

YOUR MANDATORY WORKFLOW — EVERY SESSION

Step	Command / Action
1	git pull (always first — never skip)
2	git checkout -b manny/<short-description>
3	Edit code, test with npm run dev:3000, iterate
4	git commit, then git push (pre-push hooks run ~25 sec — never use --no-verify)
5	gh pr create, then ping Sean to review. Never push to main directly.

PRE-PUSH HOOKS (NEVER BYPASS)

Six checks run automatically on every push: dynamic-pages, postgres-migrations, schema-sync, `npm test` (38 scenarios + 112K fuzz), check-secrets, push-size. If one fails, fix the underlying issue — **never use** `--no-verify`.

CRITICAL RULES

Rule	Detail
force-dynamic on every page	Every file under src/app/(app)/ must have: export const dynamic = 'force-dynamic' Without it the page cras
schema.prisma stays sqlite	provider = "sqlite" must never be changed — Amplify patches it to postgresql at build time.
Leave the budget engine alone	src/lib/calculations.js is validated. Do not touch it unless a task explicitly says to.
Auth on every API route	Every route handler starts with: const { user, error } = await requireAuth(request)

LOCAL DEV LOGIN

```
npm run dev:3000

Email:    admin@ilcstudios.com
Password: ILCadmin2024!
```

CURRENT STATE (as of 2026-07-06)

Tasks 1–5 from your last sprint all merged to main on 2026-06-24: wizard dirty-tracking, section manual adders, activity feed filters, old-to-new activity values, budget change history. Great work. Now on to Tasks 1–4 in this document — all CRM improvements, no engine or schema changes.

Important change since your last session: Mike pushed a large deploy that unified the Deal model into Project. `DealCard`, `DealDrawer`, and the activities API are gone. The pipeline entity is now `Project`. Status values live on `procurementStatus` and `designStatus` fields directly on the project. Valid statuses: `OPEN WON LOST COMPLETED ABANDONED`.

KEY FILES FOR THIS SPRINT

File	Purpose
src/app/(app)/crm/CRMClient.jsx	All four tasks live here
src/lib/pipeline.js	Pipeline helpers (statusFieldFor, stageFieldFor, etc.) — already imported in CRMC
src/app/(app)/crm/page.jsx	Model for Task 3's new Brief page — same auth + Prisma pattern
src/app/api/crm/notes/route.js	Notes API — read this before Task 4
.claude/manny-tasks.md	Live task queue — Claude reads this at session start
.claude/coordination.md	Cross-team log — read the top 40 lines

HOW TO START A SESSION

Open a terminal in the `ilc-lighting-app` folder. Run `claude` to start Claude Code. Paste the full text of `.claude/commands/formanuel.md` as your first message — it tells Claude to read the task queue and orient itself. Claude will greet you and tell you exactly what to do first.

Tasks 1–4 overview

All four tasks are CRM improvements. No schema changes, no engine changes. Work in order: 1, then 2, then 3, then 4. Tasks 1 and 2 are small and can share a session.

Task	Branch	File(s)	Effort
1 — Needs Attention filter	manny/crm-needs-attention	CRMClient.jsx	~2 hr
2 — Enhance stats bar	manny/crm-stats-enhance	CRMClient.jsx	~1 hr
3 — Morning Brief page	manny/crm-morning-brief	crm/brief/page.jsx + BriefClient.jsx	~3-4 hr
4 — Project quick-note	manny/crm-project-quicknote	CRMClient.jsx	~2 hr

Tasks 1 and 2 can be done in the same session (~3 hours together). Task 2 uses a `daysSince()` helper that Task 1 adds — do Task 1 first, or add the helper yourself if you tackle Task 2 first.

Task 1 — "Needs Attention" filter

Branch: `manny/crm-needs-attention` | File: `src/app/(app)/crm/CRMClient.jsx` | ~2 hours

Mike needs a one-click view of every open project that hasn't been touched in 30+ days so nothing slips through. Right now you have to scan the kanban card by card.

Architecture notes — read CRMClient.jsx first

* Pipeline entity is `Project` — state array is `projects`.
* Current pipeline's status field: `project[statusFieldFor(pipeline)]` returns `procurementStatus` or `designStatus`. Both helpers are already imported from `@/lib/pipeline`.
* There is **no** `daysSince()` function in the file — you must add one.
* Staleness is based on `project.updatedAt` (no activities array).
* The `filtered` useMemo is at line 294. The header/filter controls render at lines 344–392.

Step 1 — add daysSince() helper (~line 37, near other formatters)

```
function daysSince(dateStr) {
  if (!dateStr) return 999;
  return Math.floor((Date.now() - new Date(dateStr).getTime()) / 86_400_000);
}
```

Step 2 — add state inside CRMClient

```
const [needsAttention, setNeedsAttention] = useState(false);
```

Step 3 — add a derived list via useMemo

```
const needsAttentionFiltered = useMemo(() => {
  const sf = statusFieldFor(pipeline);
  return projects
    .filter(p => inBoard(p) && p[sf] === 'OPEN' && daysSince(p.updatedAt) >= 30)
    .sort((a, b) => new Date(a.updatedAt) - new Date(b.updatedAt));
}, [projects, pipeline]);
```

Step 4 — add button in the filter row (after the status pills, ~line 380)

```
<button
  onClick={() => setNeedsAttention(n => !n)}
  className={`px-2.5 py-1.5 text-[10px] font-bold rounded-lg transition-colors ${
    needsAttention
      ? 'bg-amber-500 text-white'
      : needsAttentionFiltered.length > 0
        ? 'border border-amber-400 text-amber-600 hover:bg-amber-50'
        : 'text-slate-400 hover:bg-slate-200'
  }`}
>
  {needsAttention
    ? `Needs Attention (${needsAttentionFiltered.length})`
    : needsAttentionFiltered.length > 0
      ? `! Needs Attention (${needsAttentionFiltered.length})`
      : 'Needs Attention'}
```

```
</button>
```

Step 5 — override the active list and view mode when filter is on

```
const activeList     = needsAttention ? needsAttentionFiltered : filtered;
const activeViewMode = needsAttention ? 'list' : viewMode;

// Use activeList and activeViewMode in JSX instead of filtered and viewMode.
```

Acceptance criteria

* Button shows amber outline when stale projects exist, grey when zero.
* Clicking it switches to list view showing only open + stale projects, most stale first.
* Clicking again restores normal view.
* Switching PROCUREMENT / DESIGN tab re-runs the filter for the new pipeline.
* Existing filters still work when Needs Attention is off.
* Pre-push hooks pass.

Task 2 — Enhance the existing stats bar

Branch: `manny/crm-stats-enhance` | File: `src/app/(app)/crm/CRMClient.jsx` | ~1 hour

> **Don't build a new stats bar.** One already exists: the `PipelineSummary` component at line 150 of CRMClient.jsx. It already shows 5 stat chips. Make two targeted improvements to it.

Change 1 — Replace 'Won (all time)' with 'Won (30 days)'

```
// Before:
const won = projects.filter(p => p[sf] === 'WON');

// After:
const won = projects.filter(p => p[sf] === 'WON' && daysSince(p.updatedAt) <= 30);

// Also update the label string from 'Won (all time)' to 'Won (30 days)'.
```

Change 2 — Add a 'Needs Attention' count chip

```
const stale = open.filter(p => daysSince(p.updatedAt) >= 30);

// Add to the stats array:
{
  label: 'Needs Attention',
  value: stale.length,
  color: stale.length > 0 ? 'text-amber-600' : 'text-slate-400'
}
```

Note on daysSince: This helper is added by Task 1. If you're doing Task 2 first, define it yourself (same definition as Task 1's Step 1). Only one definition should exist in the file.

Acceptance criteria

* Stats bar shows 6 chips. 'Won (30 days)' replaces 'Won (all time)'.
* 'Needs Attention' chip is amber text when count > 0, grey when 0.
* Counts update instantly when switching PROCUREMENT / DESIGN tab.
* Pre-push hooks pass.

Task 3 — Morning Brief page

Branch: `manny/crm-morning-brief` | ~3-4 hours | Do after Tasks 1 and 2.

Create a page at `/crm/brief` that gives Mike a single daily view: stale projects needing outreach, pipeline counts by stage, and recent wins.

Files to create

File	Type	Notes
src/app/(app)/crm/brief/page.jsx	Server component	MUST have: export const dynamic = 'force-dynamic'
src/app/(app)/crm/brief/BriefClient.jsx	Client component	'use client' at top; display-only, no data fetching

Model this on `src/app/(app)/crm/page.jsx` — same auth pattern, same Prisma import. You can import `PIPELINE_STAGES` from `@/lib/pipeline` — it's SSR-safe (no 'use client').

Section 1 — Needs Attention (stale open projects, top 10)

```
const thirtyDaysAgo = new Date(Date.now() - 30 * 24 * 60 * 60 * 1000);

const stale = await prisma.project.findMany({
  where: {
    OR: [{ procurementStatus: 'OPEN' }, { designStatus: 'OPEN' }],
    updatedAt: { lt: thirtyDaysAgo },
  },
  orderBy: { updatedAt: 'asc' },
  take: 10,
  select: {
    id: true, name: true, updatedAt: true,
    procurementStage: true, designStage: true,
    procurementStatus: true, designStatus: true,
    client: { select: { name: true } },
  },
});
```

Display as a table: Project | Client | Stage | Days since update. If empty: show a green 'All caught up — no stale open projects.'

Section 2 — Pipeline snapshot (open project counts by stage)

```
const [procSnapshot, designSnapshot] = await Promise.all([
  prisma.project.groupBy({
    by: ['procurementStage'],
    where: { procurementStatus: 'OPEN' },
    _count: { id: true },
  }),
  prisma.project.groupBy({
    by: ['designStage'],
    where: { designStatus: 'OPEN' },
    _count: { id: true },
  }),
]);
```

Display as two side-by-side tables (Procurement | Design), each showing Stage | Count. Use `PIPELINE_STAGES.PROCUREMENT` and `PIPELINE_STAGES.DESIGN` to control stage order and labels — don't hard-code strings.

Section 3 — Won last 30 days

```
const recentWins = await prisma.project.findMany({
  where: {
    OR: [{ procurementStatus: 'WON' }, { designStatus: 'WON' }],
    updatedAt: { gte: thirtyDaysAgo },
  },
  orderBy: { updatedAt: 'desc' },
  select: {
    id: true, name: true, updatedAt: true,
    procurementStatus: true, designStatus: true,
    client: { select: { name: true } },
  },
});
```

Display: project name, client, which pipeline(s) are Won, date updated.

Fetch all three with Promise.all. Pass stale, procSnapshot, designSnapshot, and recentWins as props to BriefClient.

Also add a 'Brief ->' link in CRMClient.jsx's header near the 'New Project' button (lines 359–368) pointing to `/crm/brief`.

Acceptance criteria

* `/crm/brief` loads without errors when logged in.
* Auth gates the page (same pattern as crm/page.jsx).
* All three sections render. Empty states handled gracefully.
* `export const dynamic = 'force-dynamic'` is on the page.
* 'Brief ->' link appears in the CRM header.
* Pre-push hooks pass.

Task 4 — Project quick-note from the kanban card

Branch: `manny/crm-project-quicknote` | File: `src/app/(app)/crm/CRMClient.jsx` | ~2 hours | Do after Tasks 1–3.

Add a '+ Note' link at the bottom of each ProjectCard so Mike can log a quick note from the kanban without navigating away from the board.

> **The old activities API is gone.** Use the new Notes API instead: `POST /api/crm/notes`. Read `src/app/api/crm/notes/route.js` before writing anything.

Notes API — POST /api/crm/notes

Minimal body to create a project-linked note:

```
{
  "noteBody": "the note text",
  "projectIds": ["project-id-here"],
  "title": "",
  "meetingDate": null,
  "attendees": "[]",
  "calendarRef": null,
  "clientIds": [],
  "activityType": null,
  "contactIds": []
}
```

Response: 201 Created on success. 4xx with { error: '...' } on failure.

NoteInput component (add above ProjectCard in the file)

```
function NoteInput({ projectId, onClose }) {
  const [text,   setText]   = useState('');
  const [saving, setSaving] = useState(false);
  const [saved,  setSaved]  = useState(false);
  const [err,    setErr]    = useState(null);

  async function handleSave() {
    if (!text.trim()) return;
    setSaving(true); setErr(null);
    try {
      const res = await fetch('/api/crm/notes', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          noteBody: text.trim(), projectIds: [projectId],
          title: '', meetingDate: null, attendees: '[]',
          calendarRef: null, clientIds: [],
          activityType: null, contactIds: [],
        }),
      });
      if (!res.ok) throw new Error((await res.json()).error || 'Failed');
      setSaved(true);
      setTimeout(onClose, 1500);
    } catch (e) {
      setErr(e.message);
```

```
    } finally {
      setSaving(false);
    }
  }

  if (saved) return <p className='text-xs text-green-600 mt-2'>Saved!</p>;

  return (
    <div className='mt-2 flex flex-col gap-1'>
      <input autoFocus value={text}
        onChange={e => setText(e.target.value)}
        placeholder='Quick note...'
        className='text-xs border border-slate-200 rounded px-2 py-1 w-full
                   focus:outline-none focus:ring-1 focus:ring-orange-400' />
      {err && <p className='text-xs text-red-500'>{err}</p>}
      <div className='flex gap-1'>
        <button onClick={handleSave}
          disabled={saving || !text.trim()}
          className='text-xs px-2 py-1 bg-slate-800 text-white rounded
                     disabled:opacity-40'>
          {saving ? 'Saving...' : 'Save'}
        </button>
        <button onClick={onClose}
          className='text-xs text-slate-400 hover:text-slate-600'>x</button>
      </div>
    </div>
  );
}
```

Add to ProjectCard's JSX (at the bottom of the return)

```
const [noteOpen, setNoteOpen] = useState(false);

// In JSX:
{!noteOpen ? (
  <button onClick={() => setNoteOpen(true)}
    className='text-xs text-slate-400 hover:text-slate-600 mt-2 w-full text-left'>
    + Note
  </button>
) : (
  <NoteInput
    projectId={project.id}
    onClose={() => setNoteOpen(false)}
  />
)}
```

Acceptance criteria

* Clicking '+ Note' opens an inline input on the card.
* Save is disabled when input is empty or while saving.
* Submitting POSTs to /api/crm/notes with projectIds: [project.id].
* Success shows 'Saved!' then closes after 1.5 seconds.
* Error message stays visible; input stays open so the user can retry.
* Pre-push hooks pass.

Operating reminder. Tell Claude what you want, not how. Claude handles git, tests, and pushes. You review and approve. Never push to main directly — always via a manny/* branch and PR. If something is unclear after 30 minutes, ask Sean.